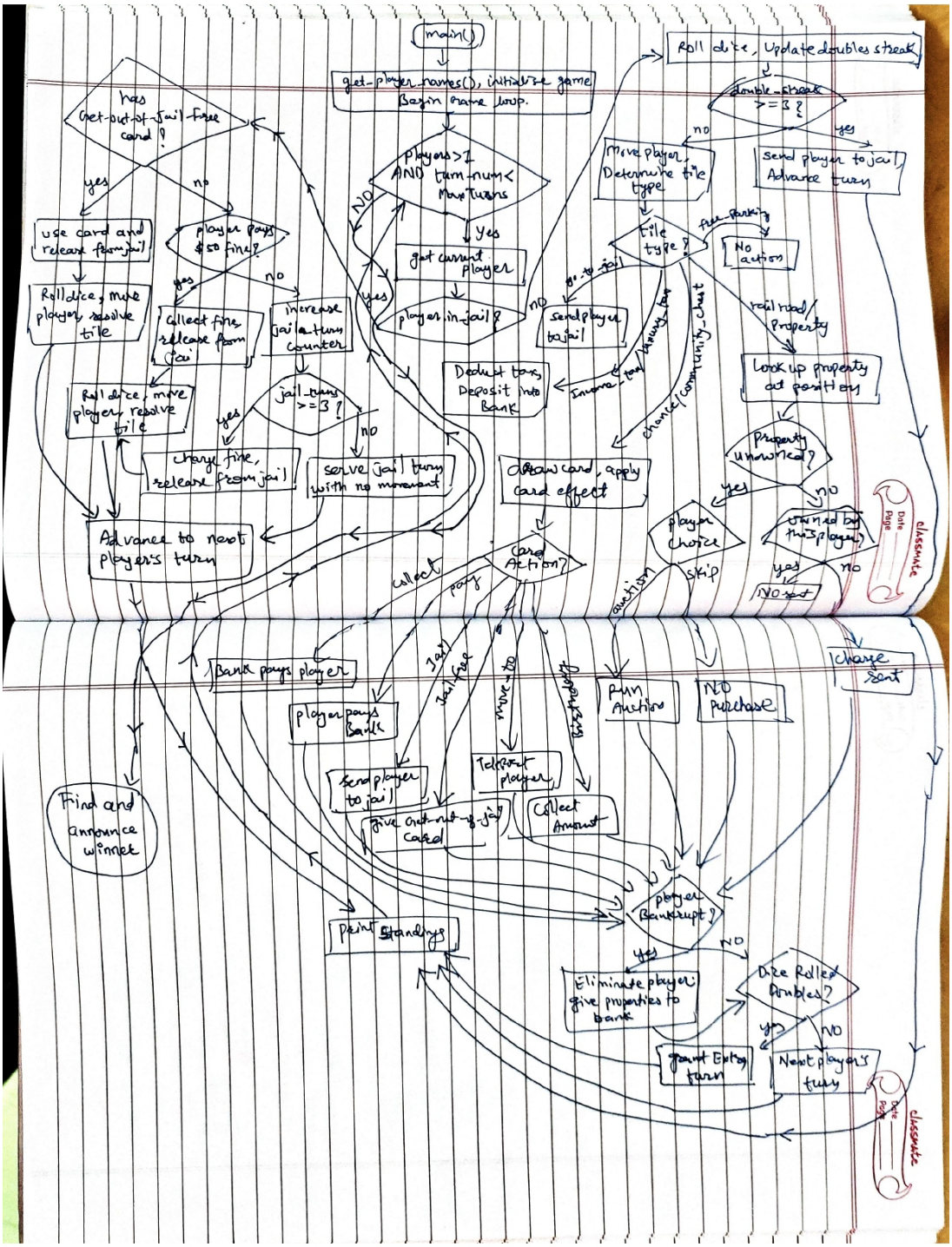
White Box Testing

1 Control Flow Graph (1.1)



2 Code Quality Analysis – pylint (1.2)

2.1 Before Fixes

Initial pylint score: **8.17 / 10**

2.2 Iteration 1: Docstrings, Unused Imports, Code Style

Changes made:

- Added module-level docstrings to all 9 files (`bank`, `board`, `cards`, `config`, `dice`, `game`, `player`, `property`, `ui`)
- Added class docstring to `Bank` and `PropertyGroup`
- Removed unused imports: `import math` (bank.py), `import sys` (player.py), `BOARD_SIZE` (dice.py), `GO_TO_JAIL_POSITION` and `import os` (game.py)
- Fixed `W0201`: moved `doubles_streak` initialisation into `Dice.__init__` (previously only in `reset()`)
- Fixed `R1705`: removed unnecessary `else` after `return` in `Property.unmortgage()`
- Fixed `C0121`: `prop.is_mortgaged == True` → `prop.is_mortgaged` (board.py)
- Fixed `W1309`: removed f-string without interpolation in `game.py` (`f"GAME OVER"` → `"GAME OVER"`)
- Fixed `R1723`: removed unnecessary `elif` after `break` in `interactive_menu()`
- Fixed `C0325`: removed superfluous parentheses after `not` in `_menu_trade()` (2 occurrences)
- Fixed `W0702`: replaced bare `except:` in `ui.safe_int_input()` with `except (ValueError, EOFError):`

Score after Iteration 1: 8.63 / 10

2.3 Iteration 2: Long Lines and Missing Newlines

Changes made:

- Reformatted all 25 long-line violations in `cards.py` (card dictionary literals split across two lines each)
- Added missing final newlines to `player.py` and `game.py` (`C0304`)

Score after Iteration 2: 9.04 / 10

2.4 Remaining Issues (Acceptable)

The remaining pylint messages are either false positives or intentional design choices:

- **E0401 import-error** – false positive: pylint is run from inside the `moneypoly/` package directory; the package resolves correctly at runtime from the parent folder.
- **R0902 too-many-instance-attributes** – `Game`, `Player`, and `Property` have 8–9 attributes each. This is an inherent requirement of a board-game domain model; refactoring would reduce clarity.

- **R0912 too-many-branches** – `_apply_card()` handles 7 distinct card action types, each a single `elif` block. Collapsing these into a dispatch dict would obscure intent.
- **R0913/R0917 too-many-arguments** – `Property.__init__` takes 5 domain-meaningful parameters; introducing a dataclass would be over-engineering for this scope.

Final pylint score: 9.04 / 10

3 White Box Test Cases (1.3)

Tests are located in `whitebox/tests/test_moneypoly.py`.
Run with: `python -m pytest whitebox/tests/ -v`
Total: **86 tests, 86 passed**.

3.1 Test Cases by Module

Class	Test	Why Needed
`TestPlayerMove`	`test_wrap_around_board`	Verifies Go salary on pass-through
	`test_land_exactly_on_go`	Verifies Go salary on landing
	`test_no_go_salary_short_move`	No false salary on short move
`TestPlayerBankruptcy`	`test_zero_balance_is_bankrupt`	Boundary: 0 counts as bankrupt
	`test_negative_balance_is_bankrupt`	Below zero is bankrupt
`TestBank`	`test_pay_out_exceeds_funds_raises`	Bank can't lend what it doesn't have
	`test_collect_negative_reduces_funds`	Mortgage payout reduces reserves
`TestDice`	`test_roll_range`	200 rolls all produce values 2–12
	`test_doubles_streak_increments`	Three-doubles-in-a-row detection
`TestPropertyGroup`	`test_all_owned_by_false_partial`	Partial group ownership must not double rent
`TestCardDeck`	`test_draw_cycles`	Deck wraps back to card 0 after exhaustion
	`test_empty_deck_returns_none`	No crash on empty deck
`TestGameBuyProperty`	`test_buy_fails_insufficient_funds`	Exact-balance purchase (documents Bug 3)
`TestGameFindWinner`	`test_find_winner_returns_richest`	Rich player must win (documents Bug 2)
`TestGameCheckBankruptcy`	`test_current_index_wraps_after_removal`	Index out-of-bounds guard

3.2 Bugs Found and Fixed

3.2.1 Bug 1: Dice rolls only 1–5 (dice.py)

Location: `moneypoly/dice.py`, `roll()` method
Found by: `TestDice::test_roll_range` (200 iterations)

Description: `random.randint(1, 5)` produces values 1–5 only. A standard die has 6 faces; totals of 11 and 12 were impossible, biasing all game outcomes.
Fix: Changed to `random.randint(1, 6)`.
Commit: `Error 1: Fix dice roll range ...`

3.2.2 Bug 2: find_winner returns the poorest player (game.py)

Location: `moneypoly/game.py`, `find_winner()`
Found by: `TestGameFindWinner::test_find_winner_returns_richest`
Description: `min(self.players, key=lambda p: p.net_worth())` returns the player with the *lowest* balance. The game should declare the richest player the winner.
Fix: Changed `min` to `max`.
Commit: `Error 2: Fix find_winner to use max() ...`

3.2.3 Bug 3: buy_property rejects exact-balance purchases (game.py)

Location: `moneypoly/game.py`, `buy_property()`
Found by: `TestGameBuyProperty::test_buy_fails_insufficient_funds`
Description: The guard `if player.balance <= prop.price` prevents a player from buying when they have *exactly* the right amount of money. The condition should use strict less-than.
Fix: Changed `<=` to `<`.
Commit: `Error 3: Fix buy_property affordability check ...`

3.2.4 Bug 4 (Fixed in Iteration 1): PropertyGroup.all_owned_by used `any` (property.py)

Location: `moneypoly/property.py`, `PropertyGroup.all_owned_by()`
Found by: `TestPropertyGroup::test_all_owned_by_false_partial`
Description: `return any(p.owner == player for p in self.properties)` returns `True` if *any* property in the group is owned by the player, causing doubled rent even for partial ownership. The check must be `all()`.
Fix: Changed `any` to `all`.
Commit: `Iteration 1: ...`

3.2.5 Bug 5 (Fixed in Iteration 1): Player.move only awards Go salary on landing, not passing (player.py)

Location: `moneypoly/player.py`, `move()`
Found by: `TestPlayerMove::test_wrap_around_board`
Description: `if self.position == 0:` only fires if the player lands *exactly* on Go. Any move that passes Go (e.g. position $38 + 4 = 2$) collected no salary.
Fix: Changed to `if self.position < prev_position or self.position == 0:`.
Commit: `Iteration 1: ...`